



NATIONAL ECONOMICS UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS

DPV

| Data Preparation & Visualization

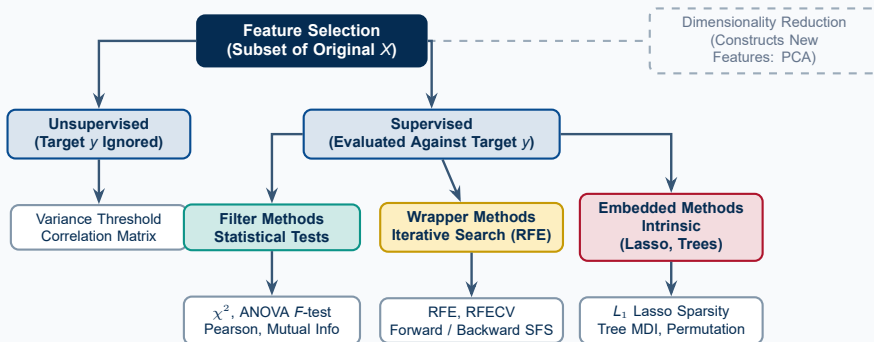
Feature Selection

Lesson 08: Filter Methods, Wrappers, Embedded Models & Pipelines

- **Instructor:** Dr. Nguyen Tuan Long
- **Academic Term:** Fall Semester — 2026-2027



Overview: Feature Selection Taxonomy



Pedagogical Sequence Across All 4 Modules: Methodological Progression

Concept → Toy Dataset → Hand Calculation → Scikit-Learn Tool → Implementation → Full-Data Lab Practice.

Doc Guide: [User Guide: Feature Selection Overview](#)

Part I: Fast Univariate Screening

Module 1: Statistical Filter Methods

- Input vs. Output variable decision matrix
- Categorical in/out: χ^2 & Mutual Information
- Numerical in / Categorical out: ANOVA F -test
- Numerical in/out: Pearson r & F -Regression
- Hands-on Lab 1: Breast Cancer & Pima Diabetes

Module 2: Wrapper Methods & Search

- Forward, Backward & Stepwise Selection
- Recursive Feature Elimination (RFE)
- Cross-Validated RFE (RFECV)
- Sequential Feature Selector (SFS)
- Hands-on Lab 2: Finding optimal k^* via CV

Part II: Integrated Optimization

Module 3: Embedded (Intrinsic) Methods

- Regularization & L_1 Lasso Sparsity
- Tree-based Gini Impurity reduction (MDI)
- Permutation Importance (Model-Agnostic truth)
- Cardinality bias in Tree MDI
- Hands-on Lab 3: Tree MDI vs Permutation

Module 4: Hybrid Pipelines & Production

- The 3 Paradigms in contrast: Speed vs Overfit
- The Multicollinearity and Cardinality Traps
- Zero Leakage Guardrail: Split First, Select Once
- Grand Pipeline: ColumnTransformer integration
- Executive Decision Framework & Takeaways



MODULE OUTLINE

Module 1: Statistical Filter Methods

Univariate Scoring, Contingency Tables & ANOVA

Motivation: Why Filter Methods?



The Big Data Reality: High Throughput Screening

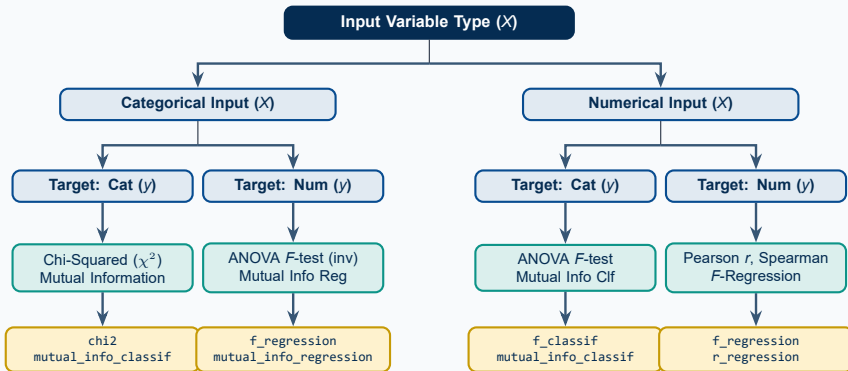
- **Speed & Scalability:** Filter methods evaluate features independently of any machine learning model in $O(N \cdot P)$ time.
- **Model-Agnosticism:** The rankings are purely statistical properties of the data, valid for any downstream estimator.
- **Initial Triage:** In genomics, finance, or sensor logs with 10^4 features, filters eliminate 90% of noise before training begins.

Doc Guide: [User Guide: Univariate Feature Selection](#)

[IMPORTANT WARNING] The Fundamental Limitation: Univariate Blindness

- **Zero Interaction Detection:** Filters cannot detect XOR relationships where two features are individually useless but together predictive.
- **Redundancy Blindness:** If X_1 and X_2 are perfectly correlated and both correlate with y , a filter selects **both**, bloating the model.
- **Recommendation:** Use filters as the first stage of a multi-stage selection funnel.

The Filter Method Decision Matrix



Doc Guide: [Scikit-Learn User Guide: Univariate Feature Selection](#)

Filter Technique 1: Chi-Squared (χ^2) Test

Mathematical Foundation

The Chi-Squared test assesses whether the observed frequency distribution of a categorical feature differs significantly from the expected distribution under independence:

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where O_{ij} is observed count in cell (i, j) , and E_{ij} is expected count:

$$E_{ij} = \frac{\text{Row Total}_i \times \text{Column Total}_j}{\text{Grand Total } N}$$

Doc Guide: [API Reference: `sklearn.feature\_selection.chi2`](#)

[IMPORTANT WARNING] Critical Assumptions & Traps

- **Non-negative Input:** Scikit-learn's `chi2` requires non-negative values ($X \geq 0$). It is inapplicable if features contain negative values from standardization.
- **Expected Frequency Rule:** At least 80% of cells must have $E_{ij} \geq 5$. Small sample counts yield uncalibrated p -values.
- **Degrees of Freedom:** $df = (r - 1)(c - 1)$. Higher cardinality features naturally accumulate larger χ^2 sums due to increased degrees of freedom.



Chi-Squared: Toy Dataset & Hand Calculation

Contingency Table: Loan Approval (y) by Education (X)

Education (X)	Default (y = 1)	Paid (y = 0)	Total
High School	30 ($E = 20$)	10 ($E = 20$)	40
Graduate	10 ($E = 20$)	30 ($E = 20$)	40
Total	40	40	$N = 80$

Expected Value Calculation

$$E_{\text{HS, Def}} = \frac{40 \times 40}{80} = 20, \quad E_{\text{HS, Paid}} = 20$$

Hand Calculation of χ^2 Statistic

Compute squared deviation over expected frequency:

$$\chi^2 = \sum \frac{(O - E)^2}{E} = 4 \times \frac{10^2}{20} = 20.0$$

[IMPORTANT WARNING] Statistical Inference

With $df = (2 - 1)(2 - 1) = 1$, the critical threshold at $\alpha = 0.001$ is $\chi^2_{\text{crit}} = 10.83$. Since $20.0 \gg 10.83$, we strongly reject independence ($p < 0.0001$).

In-Class Drill: Chi-Squared (χ^2) Contingency Table

Problem Statement: Churn by Plan ($N = 60$)

Contingency table of Subscription Plan (X) vs. Churn (y):

Plan (X)	Churn ($y = 1$)	Active ($y = 0$)	Total
Basic	20 ($E_{11} = \dots$)	10 ($E_{12} = \dots$)	30
Premium	10 ($E_{21} = \dots$)	20 ($E_{22} = \dots$)	30
Total	30	30	$N = 60$

Tasks for the Class (2 minutes):

1. Compute expected cell counts: $E_{ij} = \frac{R_i \times C_j}{N}$.
2. Compute $\frac{(O-E)^2}{E}$ for each of the 4 cells.
3. Sum to find total test statistic χ^2 .

Student Worksheet: Fill in Chi-Squared Cells

Expected count for all cells: $E = \frac{30 \times 30}{60} = \dots$

Cell (i, j)	Obs (O)	Exp (E)	$(O - E)^2$	$\frac{(O-E)^2}{E}$
Basic, Churn	20	?	?	?
Basic, Active	10	?	?	?
Prem, Churn	10	?	?	?
Prem, Active	20	?	?	?
Total χ^2	—	—	—	?

Guided Reflection: Critical Threshold Comparison

- Degrees of freedom $df = (2 - 1)(2 - 1) = 1$. Critical $\chi^2_{0.05} = 3.84$.
- Does calculated χ^2 exceed 3.84? Is Subscription Plan selected?
- Detailed step-by-step solutions are in Master Solution Manual.

Chi-Squared: Tool and Implementation

Scikit-learn API: SelectKBest(chi2)

- `score_func=chi2`: Returns test statistics and *p*-values for non-negative categorical/frequency features.
- `k=4`: Retains the top 4 highest scoring features.
- `get_feature_names_out()`: Retains column names for Pandas 2.0+ integration.

[IMPORTANT WARNING] Zero Deprecation Notice

Always ensure input features are non-negative. If continuous data contains negative values, apply `MinMaxScaler()` before `chi2`.

Doc Guide: [API: SelectKBest](#) | [API: chi2](#)

```
import numpy as np, pandas as pd
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler

# 1. Toy data: Non-negative binary indicators
X = np.array([[1, 0, 1], [1, 0, 0], [0, 1, 1], [0, 1, 0], [1, 0, 1]])
y = np.array([1, 1, 0, 0, 1])

# 2. Fit Chi-Squared Selector
selector = SelectKBest(score_func=chi2, k=2)
X_new = selector.fit_transform(X, y)
print("Chi2 Scores:", np.round(selector.scores_, 2)) # [1.33, 1.33, 0.2 ]
print("Selected feature mask:", selector.get_support()) # [ True, True, False]
```

Mutual Information for Classification

Concept: Information Theoretic Dependency

Mutual Information ($I(X; Y)$) quantifies the reduction in uncertainty of Y given knowledge of X :

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)}$$

- $I(X; Y) = 0 \iff X$ and Y are independent.
- Unlike χ^2 or Pearson r , Mutual Information captures non-linear dependencies without parametric assumptions.

Implementation: `mutual_info_classif`

Uses k -nearest neighbors entropy estimation:

Doc Guide: [API: `mutual\_info\_classif`](#)

```
from sklearn.feature\selection import mutual_info_classif

# Non-linear signal: y = sign(x^2 - 1)
X_n1 = np.random.uniform(-2, 2, (500, 1))
y_n1 = (X_n1[:, 0]**2 > 1).astype(int)

# Pearson correlation is near zero (r ~ 0.0), while MI detects
# dependency
mi = mutual_info_classif(X_n1, y_n1, random_state=42)
print("Mutual Information Score:", round(mi[0], 4)) # ~ 0.58
```

Filter Technique 2: ANOVA F-test

Numerical Input → Categorical Target

Analysis of Variance (ANOVA) tests whether the mean of numerical feature X differs significantly across target classes $k = 1, \dots, K$:

$$F = \frac{\text{Between-Group Variance}}{\text{Within-Group Variance}} = \frac{MS_{\text{between}}}{MS_{\text{within}}}$$

$$MS_{\text{between}} = \frac{\sum_{k=1}^K n_k (\bar{x}_k - \bar{x})^2}{K - 1}$$

$$MS_{\text{within}} = \frac{\sum_{k=1}^K \sum_{i=1}^{n_k} (x_{ki} - \bar{x}_k)^2}{N - K}$$

Doc Guide: [API: sklearn.feature_selection.f_classif](#)

[IMPORTANT WARNING] Geometric Intuition

- **Large F -value:** Cluster centroids are well-separated relative to intra-cluster variance, indicating strong discriminative power.
- **Small F -value ($F \approx 1$):** The variation between group means is no larger than random noise inside the groups.
- **Linear Assumption:** Assumes normal distributions within each class and equal variances (homoscedasticity).

ANOVA F-test: Toy Dataset & Hand Calculation



Toy Data: Account Balance (X) by Churn (y)

Class	Observations (x_{ki})	Mean $\bar{x}_k$	Var s_k^2
Retained ($y = 0$)	[100, 120, 110]	110.0	100.0
Churned ($y = 1$)	[20, 40, 30]	30.0	100.0
All ($N = 6$)		$\bar{x} = 70.0$	

Between-Group Sum of Squares (SS_B)

$$SS_B = 3(40)^2 + 3(-40)^2 = 4800 + 4800 = 9600$$

$$MS_B = SS_B / (2 - 1) = 9600 / 1 = 9600$$

Within-Group Sum of Squares (SS_W)

$$SS_{W,0} = (-10)^2 + 10^2 + 0^2 = 200, \quad SS_{W,1} = 200$$

$$SS_W = 200 + 200 = 400, \quad MS_W = 400 / 4 = 100$$

Final F-Ratio Calculation

$$F = \frac{MS_B}{MS_W} = \frac{9600}{100} = 96.0$$

$F = 96.0$ yields $p = 0.0006$, demonstrating clear statistical separation across churn classes.

In-Class Drill: One-Way ANOVA F-test by Hand

Problem Statement: Volume (X) by Branch ($N = 6$)

Daily volume (\$k) across $K = 3$ branches (2 days each):

- Branch 1: $[10, 20] \Rightarrow \bar{x}_1 = 15, n_1 = 2$
- Branch 2: $[30, 50] \Rightarrow \bar{x}_2 = 40, n_2 = 2$
- Branch 3: $[60, 70] \Rightarrow \bar{x}_3 = 65, n_3 = 2$

Overall grand mean: $\bar{x} = \frac{15+40+65}{3} = 40.0$.

Tasks for the Class (2 minutes):

1. Compute $SS_B = \sum_{k=1}^3 n_k(\bar{x}_k - \bar{x})^2$ and MS_B .
2. Compute $SS_W = \sum (x_{ki} - \bar{x}_k)^2$ and MS_W .
3. Compute ANOVA F -ratio: $F = MS_B/MS_W$.

Student Worksheet: Fill in ANOVA Table

Degrees of freedom: $df_B = K - 1 = 2, df_W = N - K = 3$.

Source	SS	df	MS	F-Statistic
Between (B)	$2(25^2 + 0^2 + 25^2) = \dots$	2	$SS_B/2 = \dots$	$MS_B/MS_W = ?$
Within (W)	$50 + 200 + 50 = \dots$	3	$SS_W/3 = \dots$	-
Total	$SS_B + SS_W = \dots$	5	-	-

Guided Reflection: Variance Ratio Meaning

- Is between-group variation significantly larger than within-group noise?
- Does Branch feature exhibit strong discriminative power?
- Detailed step-by-step solutions are in Master Solution Manual.

ANOVA F-test: Tool and Implementation



Scikit-learn Tool: `f_classif`

- Computes 1-way ANOVA F -values across target classes.
- Operations are vectorized across thousands of columns in a single matrix pass.

Doc Guide: [API: `sklearn.feature\_selection.f\_classif`](#)

[IMPORTANT WARNING] Integration Recommendation

Combine with `SelectPercentile` when feature dimensions vary widely, selecting the top 20% most informative features.

Doc Guide: [API: `SelectPercentile`](#)

```
from sklearn.feature_selection import SelectKBest, f_classif
import numpy as np

# Replicating toy dataset: Feature 1 (Balance), Feature 2 (Random Noise)
X = np.array([[100, 5], [120, 6], [110, 4], [20, 5], [40, 6], [30, 4]])
y = np.array([0, 0, 0, 1, 1, 1])

selector = SelectKBest(score_func=f_classif, k=1)
X_selected = selector.fit_transform(X, y)

print("Exact ANOVA F-scores:", np.round(selector.scores_, 2)) # [96.0, 0.0]
print("p-values:", np.round(selector.pvalues_, 4)) # [0.0006, 1.0]
print("Selected feature shape:", X_selected.shape) # (6, 1) -> Isolates Feature 1
```

Filter Technique 3: Numerical In → Numerical Out

Correlation & Regression Metrics

When both feature X_j and target y are continuous:

- **Pearson's r :** Measures linear correlation:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \in [-1, 1]$$

- **F-Regression (f_regression):** Tests significance of linear regression slope β_1 :

$$F = \frac{r^2 / 1}{(1 - r^2) / (N - 2)}$$

Identical feature ranking as $|r|$, but provides calibrated p -values.

Doc Guide: [API: sklearn.feature_selection.f_regression](#)

[IMPORTANT WARNING] Monotonic vs. Linear Dependencies

- **Pearson's r :** Blind to non-linear monotonic relationships (e.g. $y = \exp(x)$ or $y = x^3$).
- **Spearman's ρ :** Computes Pearson's r on ranked observations, providing robustness against outliers and monotonic non-linearities.
- **mutual_info_regression:** For complex non-monotonic surfaces (e.g. sinusoidal, bimodal curves).

Pearson Correlation & F-Regression: Hand Calculation



Toy Regression Data ($N = 4$)

i	x_i	y_i	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	1	2	$(-1.5)(-3) = 4.5$	2.25
2	2	4	$(-0.5)(-1) = 0.5$	0.25
3	3	6	$(0.5)(1) = 0.5$	0.25
4	4	8	$(1.5)(3) = 4.5$	2.25
Mean	$\bar{x} = 2.5$	$\bar{y} = 5$	$\sum = 10.0$	$\sum = 5.0$

Step-by-step r and F -score

$$\sum (y_i - \bar{y})^2 = 9 + 1 + 1 + 9 = 20.0$$

$$r = \frac{10.0}{\sqrt{5.0 \times 20.0}} = \frac{10.0}{\sqrt{100.0}} = 1.0$$

$$F = \frac{r^2(N-2)}{1-r^2} \rightarrow \infty \quad (\text{Perfect Fit})$$

[IMPORTANT WARNING] Analytical Insight

When $r = 1.0$, the feature completely explains target variance ($R^2 = 100\%$). In real datasets, ranking by $f_{\text{regression}}$ is equivalent to ranking by r^2 .

In-Class Drill: Pearson Correlation & F-Regression

Problem Statement: Ad Spend (X) vs Sales (y)

Observed $N = 3$ advertising campaigns (\$k):

$$\mathbf{x} = [1, 2, 3]^T \Rightarrow \bar{x} = 2.0$$

$$\mathbf{y} = [2, 5, 5]^T \Rightarrow \bar{y} = 4.0$$

Tasks for the Class (2 minutes):

1. Compute centered deviations: $(x_i - \bar{x})$ and $(y_i - \bar{y})$.
2. Compute cross-product sum $\sum (x_i - \bar{x})(y_i - \bar{y})$.
3. Compute Pearson $r = \frac{\sum dx \cdot dy}{\sqrt{\sum dx^2 \sum dy^2}}$.
4. Convert r to ANOVA F -statistic: $F = \frac{r^2(N-2)}{1-r^2}$.

Student Worksheet: Fill in Correlation Table

Deviations: $\mathbf{x} - 2.0 = [-1, 0, 1]^T$, $\mathbf{y} - 4.0 = [-2, 1, 1]^T$.

ID	$(x_i - \bar{x})$	$(y_i - \bar{y})$	Cross-Product	$(y_i - \bar{y})^2$
1	-1.0	-2.0	$(-1)(-2) = \dots$	$(-2)^2 = \dots$
2	0.0	+1.0	$(0)(1) = \dots$	$(1)^2 = \dots$
3	+1.0	+1.0	$(1)(1) = \dots$	$(1)^2 = \dots$
Sum	$\sum dx^2 = 2.0$	-	$\sum = ?$	$\sum = ?$

Guided Reflection: From r to F -Score

- Calculated $r = \frac{3.0}{\sqrt{2.0 \times 6.0}} = \dots \Rightarrow r^2 = \dots$
- What is the resulting F -score for $N = 3$ ($N - 2 = 1$)?
- Detailed step-by-step solutions are in Master Solution Manual.



Correlation & Regression: Tool and Implementation

```
from sklearn.feature_selection import SelectPercentile, f_regression
from sklearn.datasets import make_regression
import numpy as np

# 1. Synthetic regression problem: 100 samples, 10 features (only 3 informative)
X_reg, y_reg = make_regression(n_samples=100, n_features=10, n_informative=3, noise=5.0, random_state=42)

# 2. Select top 30% features using F-Regression
selector = SelectPercentile(score_func=f_regression, percentile=30)
X_selected = selector.fit_transform(X_reg, y_reg)

print("All F-Scores:\n", np.round(selector.scores_, 1))
print("Selected feature indices:", np.where(selector.get_support())[0])
print("Shape reduction:", X_reg.shape, "->", X_selected.shape)
```

Scikit-learn Distinction: $r_{\text{regression}}$ vs. $f_{\text{regression}}$

$r_{\text{regression}}$ returns raw signed Pearson coefficients $r \in [-1, 1]$ (preserves positive/negative direction). $f_{\text{regression}}$ returns strictly non-negative F -test scores and p -values suitable for ranking.

Doc Guide: [API: \$f_{\text{regression}}\$](#) | [API: \$r_{\text{regression}}\$](#)



[IMPORTANT WARNING] Pitfall 1: Multi-Collinear Duplication

- Suppose $X_1 = \text{Income}$ and $X_2 = \text{Tax Paid} \approx 0.3 \times \text{Income}$.
- Both exhibit large ANOVA F -statistics against loan default.
- A filter method **selects both**, introducing severe multicollinearity into linear models and wasting memory.

[IMPORTANT WARNING] Limitation 2: Non-Linear XOR Interactions

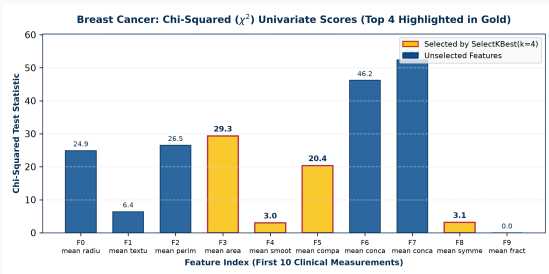
- Let $y = X_1 \oplus X_2$ (exclusive OR).
- Separately, X_1 has zero correlation with y ($r = 0, F \approx 0$).
- X_2 also has zero correlation with y ($r = 0, F \approx 0$).
- Filter methods discard **both**, preventing downstream estimators from capturing the interaction.

Managerial Takeaway: Never Rely Solely on Filter Methods

Filters are excellent first-stage sweepers, but model-aware techniques (Wrappers or Embedded) must follow.

Doc Guide: [User Guide: Univariate Feature Selection Pitfalls](#)

Hands-on Lab 1: Chi-Squared on Clinical Cancer Data

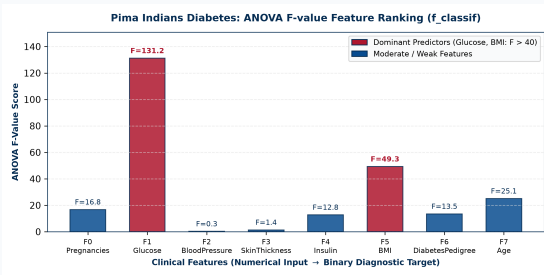


Lab Setup & Findings

Target: Malignant ($y = 1$) vs. Benign ($y = 0$).

- Features normalized with `MinMaxScaler` to ensure $X \geq 0$.
- `SelectKBest(chi2, k=4)` isolates Feature 3 (mean area, $\chi^2 = 29.4$), Feature 4 (mean smoothness), Feature 5, and Feature 8.
- **Accuracy Benchmark:** 4 selected features preserve 74.7% accuracy while reducing feature space by 60%.

Hands-on Lab 2: ANOVA F-test on Pima Diabetes



Clinical Diagnostic Insights

Target: Diabetes Onset (Binary 0/1).

- **Dominant Signal:** Feature 1 (Glucose) exhibits $F = 131.2$ ($p < 10^{-20}$), followed by Feature 5 (BMI, $F = 49.3$).
- **Noise Features:** Feature 2 (BloodPressure, $F = 0.3$) and Feature 3 (SkinThickness, $F = 1.4$) provide negligible diagnostic discrimination.
- Pruning weak features eliminates noise and improves logistic regression convergence.

Module 1 Synthesis: Filter Methods Decision Matrix



Decision Matrix for Statistical Filter Methods

Filter Test	Input Type	Output Type	Relationship Captured	Key Assumption / Constraint	Scikit-learn Tool
Chi-Squared (χ^2)	Categorical	Categorical	Contingency independence	Non-negative $X \geq 0$, expected count ≥ 5	chi2
ANOVA F -test	Numerical	Categorical	Difference in group means	Normality, equal group variance	f_classif
Pearson Correlation	Numerical	Numerical	Linear co-variation	Linear relationship, sensitive to outliers	r_regression
F -Regression	Numerical	Numerical	Linear regression slope $\neq 0$	Normality of residuals	f_regression
Mutual Information	Any	Any	Non-linear arbitrary entropy	Requires more data samples, slower $O(N \log N)$	mutual_info_classif

Executive Recommendation:

Use **ANOVA F -test** or χ^2 for fast baseline triage on large tabular sets; switch to **Mutual Information** when domain knowledge strongly suggests non-linear physical or economic thresholds.



MODULE OUTLINE

Module 2: Wrapper Methods & Search Strategies

Search Strategies, RFE, RFECV & SequentialFeatureSelector

Motivation: The Wrapper Philosophy

The Core Concept: Model in the Loop

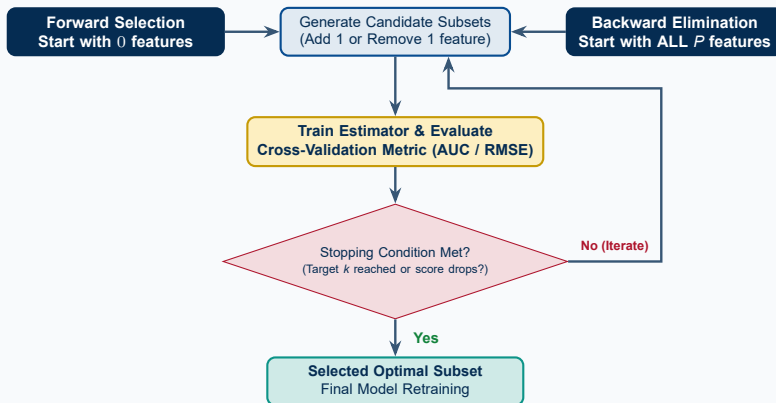
- Rather than scoring features in a vacuum, a **Wrapper** evaluates subsets of features using the actual predictive performance of an Estimator.
- **Captures Interactions:** If X_1 and X_2 exhibit synergistic interactions, validation scores improve when both are jointly included.
- **Model Specificity:** A feature subset optimized for a Decision Tree may differ from one optimized for Logistic Regression.

[IMPORTANT WARNING] The Combinatorial Dilemma

- For P features, there exist $2^P - 1$ possible subsets.
- For $P = 30$, evaluating all $2^{30} \approx 1.07 \times 10^9$ subsets makes exhaustive search computationally intractable.
- **Greedy Heuristics:** We must use heuristic search strategies: Forward Selection, Backward Elimination, or Recursive Pruning.

Doc Guide: [User Guide: Sequential Feature Selection](#)

Wrapper Search: Forward, Backward & Stepwise



Doc Guide: [Scikit-Learn User Guide: Sequential Feature Selection](#)

Recursive Feature Elimination (RFE): Mechanics

Algorithmic Workflow

RFE is a specialized, highly efficient backward elimination wrapper:

1. Train the supervised estimator on the initial set of features.
2. Obtain importance rankings:
 - ▶ Linear models: $|\text{coef}_j|$
 - ▶ Tree models: `feature_importances_`
3. Prune the step features with the smallest importance scores.
4. Repeat steps 1–3 until exactly `n_features_to_select` remain.

[IMPORTANT WARNING] Why RFE Beats Simple Ranking

- **Dynamic Coefficient Re-estimation:** When a redundant collinear feature is eliminated, remaining coefficients adjust to capture the shared variance.
- A single static ranking would discard both if individual scores appeared modest.
- **Weakness:** Requires an estimator that exposes `coef_` or `feature_importances_`.

Doc Guide: [User Guide: Recursive Feature Elimination](#)

RFE: Toy Dataset & Hand Calculation

Iteration 1: Initial Linear Model ($P = 4$)

Model fitted on all 4 standardized features:

$$\hat{y} = 3.2x_1 - 0.4x_2 + 2.1x_3 + 0.05x_4$$

Feature	Magnitude $ \beta_j $	RFE Status
x_1	3.20	Retain
x_2	0.40	Retain
x_3	2.10	Retain
x_4	0.05	ELIMINATE (Lowest)

Iteration 2: Refit on $\{x_1, x_2, x_3\}$

Refitting estimator without x_4 :

$$\hat{y} = 3.25x_1 - 0.38x_2 + 2.15x_3$$

Feature	Magnitude $ \beta_j $	RFE Status
x_1	3.25	Retain (Rank 1)
x_2	0.38	ELIMINATE (Rank 3)
x_3	2.15	Retain (Rank 2)

Final Ranking Vector:

$\text{ranking_} = [1, 2, 1, 3]$ (If target $k = 2$, selected subset is $\{x_1, x_3\}$).

In-Class Drill: RFE Step-by-Step Weight Elimination

Problem Statement: Financial Predictors ($P = 4$)

Iteration 1 (All 4 features):

$$\hat{y} = 1.80x_1 - 2.50x_2 + 0.30x_3 - 0.08x_4$$

Iteration 2 (Refit after 1st drop):

$$\hat{y} = 1.90x_1 - 2.40x_2 + 0.35x_3$$

Tasks for the Class (2 minutes):

1. Identify feature with lowest $|\beta_j|$ in Iteration 1.
2. Identify next eliminated feature in Iteration 2.
3. If target subset size $k = 2$, which features survive?
4. Complete the final Scikit-learn `ranking_` vector.

Student Worksheet: Fill in RFE Elimination Table

Magnitudes: $|1.80| = 1.8$, $|-2.50| = 2.5$, $|0.30| = 0.3$, $|-0.08| = 0.08$.

Feature	Iter 1 $ \beta $	Iter 2 $ \beta $	Status ($k = 2$)	Rank
x_1	1.80	1.90	?	?
x_2	2.50	2.40	?	?
x_3	0.30	0.35	?	?
x_4	0.08	—	?	?

Guided Reflection: Why Greedy Backward?

- Notice how β_1 and β_3 shifted after dropping x_4 (correlation readjustment).
- What would final `support_` boolean mask look like for $k = 2$?
- Detailed step-by-step solutions are in Master Solution Manual.



RFE: Tool and Scikit-Learn Implementation

```
from sklearn.feature\selection import RFE
from sklearn.linear\model import LogisticRegression
from sklearn.datasets import make_classification
import numpy as np

# 1. Synthetic dataset: 1000 samples, 10 features (5 informative, 5 noise)
X, y = make_classification(n_samples=1000, n_features=10, n_informative=5, random_state=42)

# 2. Instantiate Base Estimator & RFE
estimator = LogisticRegression(solver="lbfgs", max_iter=500, random_state=42)
rfe = RFE(estimator=estimator, n_features_to_select=5, step=1)
rfe.fit(X, y)

# 3. Inspect Rankings
print("Selected feature mask:", rfe.support_)
print("Feature ranking array:", rfe.ranking_)
# ranking_ == 1 indicates the top selected features; 2, 3... indicates elimination sequence.
```

Key Hyperparameter: **step**

step=1 eliminates one feature per refit (most thorough, P fits). step=0.1 eliminates 10% of features per refit (much faster for $P > 500$).

Doc Guide: [API: sklearn.feature_selection.RFE](#)



Automated k^* Determination

Standard RFE requires you to arbitrarily pick $k = \text{n_features_to_select}$. How do you know if $k = 5$ is better than $k = 8$?

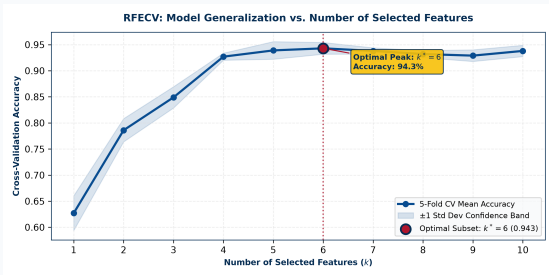
- RFECV embeds RFE inside cross-validation loops (e.g. 5-Fold Stratified CV).
- Evaluates model generalization score for every subset size $k \in [1, P]$.
- Automatically identifies the subset size k^* that maximizes out-of-fold validation performance.

[IMPORTANT WARNING] Scikit-Learn 1.4+ API Update

- **Deprecation Notice:** `rfecv.grid_scores_` has been deprecated and removed.
- **Modern Standard:** Access cross-validation results via `rfecv.cv_results_["mean_test_score"]`.
- `rfecv.n_features_`: Directly returns the optimal feature count k^* .

Doc Guide: [API: sklearn.feature_selection.RFECV](#)

RFECV: Empirical Convergence & Scoring Curve



Anatomy of the Generalization Curve

- **Underfitting Zone ($k < 4$):** Removing informative features starves the model, accuracy plummets (72% $\rightarrow$ 82%).
- **Optimal Peak ($k^* = 5$):** Accuracy peaks at 89.2%. The 5 truly informative features are retained!
- **Overfitting / Noise Zone ($k > 5$):** Adding redundant features increases variance, causing CV score to plateau or degrade.



Sequential Feature Selector: Forward vs. Backward

SequentialFeatureSelector Mechanics

Unlike RFE, SFS does not require internal model coefficients or importance attributes, supporting arbitrary estimators (e.g., KNN, kernel SVM).

- **direction="forward"**: Greedily adds the single feature that maximizes CV score.
- **direction="backward"**: Greedily removes the feature whose removal least hurts CV score.

[IMPORTANT WARNING] Computational Warning

SFS fits an estimator at every step for every candidate. For large P , forward is much faster than backward.

Doc Guide: [API: SequentialFeatureSelector](#)

```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
sfs = SequentialFeatureSelector(estimator=knn, n_features_to_select=3, direction="forward", cv=5)
sfs.fit(X, y)

print("Selected features via Forward SFS:", np.where(sfs.get_support())[0])
```

Comparison: RFE vs. Sequential Feature Selector (SFS)



Recursive Feature Elimination (RFE)

- **Mechanism:** Uses model internal coefficients ($|\beta_j|$) to rank and prune.
- **Speed:** Requires only $P - k$ model fits (or fewer if $\text{step} > 1$). Relatively fast.
- **Estimator Requirement:** Strictly limited to models exposing `coef_` or `feature_importances_`.
- **Direction:** Strictly backward elimination.

Sequential Feature Selector (SFS)

- **Mechanism:** Uses out-of-fold cross-validation score of candidate subsets.
- **Computational Complexity:** Requires $O(P \cdot k)$ estimator fits for forward selection and $O(P^2)$ fits for backward elimination.
- **Estimator Requirement:** 100% model-agnostic. Works with KNN, Naive Bayes, non-linear SVM.
- **Direction:** Both forward and backward supported.



[IMPORTANT WARNING] Hazard 1: Search Overfitting

- Evaluating extensive candidate subsets against a single validation score introduces risks of overfitting the validation split.
- A feature subset might perform well by chance on a specific 5-fold split.
- **Remedy:** Always evaluate the final selected model on a completely isolated, untouched Test set.

[IMPORTANT WARNING] Hazard 2: High Computational Latency

- Running RFECV with Random Forest (100 trees) on 50 features and 5 folds requires:

$$5 \text{ folds} \times 50 \text{ steps} = 250 \text{ model fits.}$$

- Can take hours on large tabular datasets.
- **Remedy:** Use faster linear base estimators (e.g. LogisticRegression) or set step=5.



[IMPORTANT WARNING] Methodological Error: Selection Leakage

- **Flawed Workflow:** Run RFE on the full dataset $D = (X, y)$ to select top 5 features $\rightarrow$ then run `cross_val_score` on (X_{selected}, y) .
- **The Outcome:** Selected features reflect spurious correlation with the validation target rather than genuine generalization.
- Generates unrealistically optimistic scores (e.g. 98% CV accuracy) that crater to 65% on live production data!

The Production Rule: Pipeline Encapsulation

- The wrapper selector **must** be embedded inside the Scikit-learn Pipeline.
- In every cross-validation fold, RFE is fitted strictly on the 80% training split and applied to the 20% validation split.
- Prevents selection leakage across validation folds.

Doc Guide: [Scikit-Learn Guide: Preventing Data Leakage](#)



Hands-on Lab 3: Tuning Optimal Subsets via RFECV

```
from sklearn.feature\selection import RFECV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.datasets import make_classification

# 1. Generate realistic benchmark with 20 features (5 signal, 15 noise)
X_bench, y_bench = make_classification(n_samples=800, n_features=20, n_informative=5,
                                     n_redundant=3, random_state=42)

# 2. Configure RFECV with 5-Fold Stratified CV
rf = RandomForestClassifier(n_estimators=50, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rfecv = RFECV(estimator=rf, step=1, cv=cv, scoring="accuracy", n_jobs=-1)
rfecv.fit(X_bench, y_bench)

# 3. Analyze Results
print(f"Optimal feature subset size k*: {rfecv.n_features_}")
print(f"Peak CV Accuracy: {rfecv.cv_results_['mean_test_score'].max():.4f}")
print(f"Features chosen: {np.where(rfecv.support_)[0]}")
```

Lab Takeaway:

RFECV successfully discards the 15 redundant and noise features, maintaining high accuracy with a 75% compression ratio.

Module 2 Synthesis: Wrapper Methods Decision Matrix



Decision Matrix for Wrapper Selection Methods

Wrapper Tool	Search Direction	Estimator Class	Complexity	Interaction Handling	Stopping Mechanism
RFE	Backward	Linear / Tree (with coef_)	$O(P - k)$	Dynamic re-weighting	Fixed k (n_features_to_select)
RFECV	Backward	Linear / Tree (with coef_)	$O(CV \cdot P)$	Dynamic re-weighting	Automated max CV score (k^*)
Forward SFS	Forward	100% Model-Agnostic	$O(P \cdot k)$	Captures greedy synergy	Fixed k or score tolerance
Backward SFS	Backward	100% Model-Agnostic	$O(P^2)$	Evaluates full interactions	Fixed k or score tolerance

Executive Recommendation:

For tabular datasets with $P \leq 50$, **RFECV** paired with a fast linear or tree estimator is an effective baseline. For high-dimensional datasets ($P \gg 50$), apply filter methods first to pre-screen candidates before wrapping.



MODULE OUTLINE

Module 3: Embedded Methods & Model-Based Importance

Lasso L1 Sparsity, Tree MDI & Permutation Importance



The Best of Both Worlds

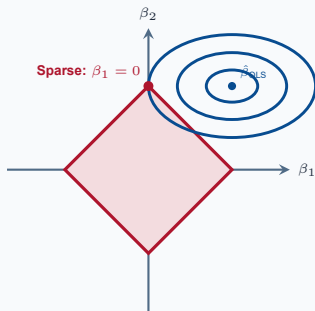
- **Integrated Selection:** Feature selection is built directly into the mathematical objective function of the machine learning algorithm.
- **Efficiency of Filters:** No external search loop is required; selection occurs directly during a single training cycle.
- **Interactions of Wrappers:** Captures complex feature interactions and non-linearities directly during tree splitting or regularized loss optimization.

Three Core Embedded Families

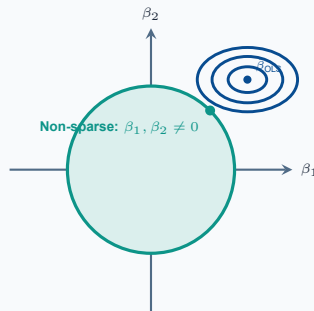
1. **Regularization (L_1 Penalty):** Lasso drives irrelevant coefficients to **exact zero**, performing sparse selection.
2. **Tree Impurity (MDI):** Random Forests aggregate Gini/variance reduction across all splits where feature j was used.
3. **Permutation Importance:** Model-agnostic inspection breaking correlations to measure accuracy drop.

Doc Guide: [User Guide: Feature Selection with SelectFromModel](#)

Regularization Geometry: L_1 Lasso vs. L_2 Ridge



Lasso (L_1 Diamond Corner)



Ridge (L_2 Smooth Circle)

Geometric Truth:

The vertices of the L_1 diamond lie on coordinate axes. Unpenalized loss contours typically make contact at these corners, setting parameters to **exact zero**.

Doc Guide: [User Guide: Lasso Regularization & Sparsity](#)

Lasso Sparsity: Mathematical Formulation

The L_1 Regularized Objective

$$\min_w \frac{1}{2N} \|y - Xw\|_2^2 + \alpha \|w\|_1$$

where $\|w\|_1 = \sum_{j=1}^P |w_j|$.

- $\alpha \rightarrow 0$: Ordinary Least Squares (all features retained).
- $\alpha \rightarrow \infty$: Maximum penalty (all $w_j = 0$).
- **Soft-Thresholding Operator:** For orthonormal design:

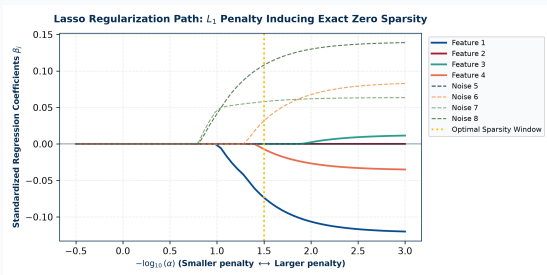
$$w_j^* = \text{sign}(\hat{w}_j^{\text{OLS}}) \max(0, |\hat{w}_j^{\text{OLS}}| - \alpha)$$

Doc Guide: [API Reference: sklearn.linear_model.LassoCV](#)

[IMPORTANT WARNING] Critical Prerequisite: Feature Scaling

- The penalty term $\alpha \sum |w_j|$ penalizes all weights equally.
- If X_1 is measured in large units ($\$10^6$) and X_2 in small units (20), their coefficient scales diverge artificially.
- Lasso penalizes coefficients irrespective of measurement units, unfairly shrinking features with large natural scales.
- **Prerequisite:** Standardize features (StandardScaler) before running Lasso.

Lasso Sparsity: Regularization Path Analysis



Interpreting the Sparsity Path

- **Right to Left ($-\log_{10} \alpha$ decreases $\Rightarrow \alpha$ increases):** As penalty stiffens, noisy features hit zero first.
- **Optimal Window ($\alpha \approx 0.03$):** Noise features 5–8 are clamped strictly at zero, while true informative features 1–4 remain non-zero.
- **Selection Tool:**
`SelectFromModel(Lasso(alpha=0.03))`
 extracts the non-zero feature subset.

Lasso Sparsity: Tool and Implementation

```
from sklearn.linear\_model import LassoCV
from sklearn.feature\_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

# 1. Pipeline with Scaler + LassoCV (auto-tunes optimal alpha via cross-validation)
lasso_cv = LassoCV(cv=5, random_state=42)
lasso_cv.fit(X_bench, y_bench)

print(f"Optimal Alpha tuned by CV: {lasso_cv.alpha_:.4f}")
print("Non-zero coefficients count:", np.sum(lasso_cv.coef_ != 0))

# 2. Extract selected features via SelectFromModel
selector = SelectFromModel(estimator=lasso_cv, prefit=True)
X_sparse = selector.transform(X_bench)
print("Compressed Feature Shape:", X_sparse.shape)
```

Scikit-learn Tool: SelectFromModel

SelectFromModel works with any model exposing coef_ or feature_importances_, selecting features exceeding a threshold (default: mean).

Doc Guide: [API: sklearn.feature_selection.SelectFromModel](#)

Tree Feature Importance: Mean Decrease in Impurity

How Random Forest Computes MDI

In Decision Trees, at every split node t using feature X_j , the impurity reduction is:

$$\Delta I(t) = I(t) - \frac{N_{t_L}}{N_t} I(t_L) - \frac{N_{t_R}}{N_t} I(t_R)$$

The MDI importance of feature j is the sum of weighted impurity decreases across all nodes and all B trees:

$$\text{MDI}_j = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b, v(t)=j} \frac{N_t}{N} \Delta I(t)$$

Key Advantages of MDI

- **Computational Efficiency:** Impurity scores are accumulated during tree training with negligible additional cost.
- **Non-linear Interactions:** Naturally credits features involved in deep branch splits.
- **No Scaling Needed:** Tree splits are invariant to monotonic transformations and scaling.

Doc Guide: [User Guide: Random Forest Feature Importance](#)

Tree MDI: Toy Dataset & Hand Calculation

Toy Split: $N = 10$ Samples (5 Pos, 5 Neg)

Root Node t : $I(t) = 1 - (0.5^2 + 0.5^2) = 0.50$.

Candidate Split	Left (t_L)	Right (t_R)	Gini Impurity
$X_1 \leq 3$	4 Pos, 0 Neg ($N_L = 4$)	1 Pos, 5 Neg ($N_R = 6$)	$I_L = 0, I_R = 0.278$
$X_2 \leq 5$	3 Pos, 2 Neg ($N_L = 5$)	2 Pos, 3 Neg ($N_R = 5$)	$I_L = 0.48, I_R = 0.48$

Computing Impurity Gain ΔI

$$\Delta I(X_1) = 0.50 - (0.4 \times 0 + 0.6 \times 0.278) = \mathbf{0.333}$$

$$\Delta I(X_2) = 0.50 - (0.5 \times 0.48 + 0.5 \times 0.48) = \mathbf{0.020}$$

MDI Attribution:

Feature X_1 yields $\Delta I = 0.333$ vs 0.020 for X_2 . X_1 receives 94.3% of the tree importance!

In-Class Drill: Decision Tree Gini MDI Reduction

Problem Statement: Loan Defaults ($N = 8$)

Root node t : 4 Defaults, 4 Paid $\implies I(t) = 0.50$.

Candidate Split A ($X_1 \leq 30$):

- Left ($N_L = 4$): 4 Defaults, 0 Paid $\implies I_L = 0.0$
- Right ($N_R = 4$): 0 Defaults, 4 Paid $\implies I_R = 0.0$

Candidate Split B ($X_2 \leq 50$):

- Left ($N_L = 4$): 3 Defaults, 1 Paid $\implies I_L = 0.375$
- Right ($N_R = 4$): 1 Default, 3 Paid $\implies I_R = 0.375$

Tasks for the Class (2 minutes):

1. Compute post-split Gini: $I_{\text{split}} = \frac{N_L}{N} I_L + \frac{N_R}{N} I_R$.
2. Compute impurity reduction: $\Delta I = I(t) - I_{\text{split}}$.
3. Determine winning feature and importance share.

Student Worksheet: Fill in MDI Comparison

Weights: $\frac{N_L}{N} = \frac{4}{8} = 0.50$, $\frac{N_R}{N} = \frac{4}{8} = 0.50$.

Candidate	I_L	I_R	Post-Split I	Gain ΔI
$X_1 \leq 30$	0.000	0.000	$0.5(0) + 0.5(0) = \dots$	$0.50 - \dots = ?$
$X_2 \leq 50$	0.375	0.375	$0.5(.375) + 0.5(.375) = \dots$	$0.50 - \dots = ?$
Winner	-	-	-	?

Guided Reflection: Gini Purity vs Split Quality

- Notice X_1 achieves perfect pure separation ($I_L = I_R = 0$).
- Why does X_1 capture 80% of total impurity decrease?
- Detailed step-by-step solutions are in Master Solution Manual.



[IMPORTANT WARNING] Theoretical Limitation of Impurity Importance

- **Cardinality Bias:** Continuous features or categorical features with many unique values (e.g. Zipcode, Customer ID) offer vastly more candidate split points.
- Even if a high-cardinality feature is **100% pure random noise**, the tree can find accidental splits that appear to reduce training impurity!
- **Outcome:** MDI can assign spuriously high importance to uninformative high-cardinality features.

The Academic Discovery:

Strobl et al. (2007) mathematically proved that standard Random Forest MDI is severely biased towards high-cardinality features. **Best Practice:** Validate tree importances using **Permutation Importance** evaluated on an out-of-fold validation set.

Doc Guide: [Guide: Permutation vs. Impurity Importance](#)



Permutation Feature Importance: Model-Agnostic Truth

Algorithmic Mechanism

1. Record baseline validation score S_{base} on validation set D_{val} .
2. For each feature j :
 - ▶ Shuffle (permute) the values of column j , breaking its relationship with target y .
 - ▶ Recompute validation score $S_{\text{perm},j}$.
 - ▶ Importance: $I_j = S_{\text{base}} - S_{\text{perm},j}$.
3. Repeat n times to obtain mean and standard deviation.

Implementation Syntax

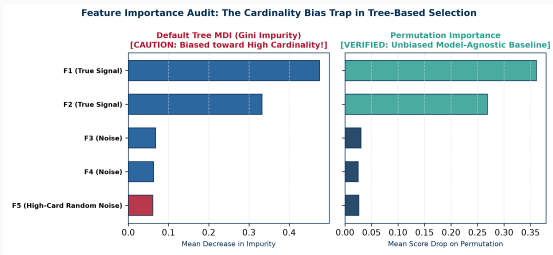
`sklearn.inspection.permutation_importance`

Doc Guide: [User Guide: Permutation Feature Importance](#)

```
from sklearn.inspection import permutation_importance

perm_res = permutation_importance(
    estimator=rf,
    X=X_val,
    y=y_val,
    scoring="accuracy",
    n_repeats=10,
    random_state=42,
    n_jobs=-1
)
print("Mean Drop:", perm_res.importances_mean)
```

Permutation Importance: Empirical Comparison vs. MDI



Evidence-Based Audit

- **Left Panel (MDI):** Feature F5 (Random High-Cardinality Noise) is spuriously ranked as a prominent predictor.
- **Right Panel (Permutation):** Permutation on validation set accurately exposes F5 as **pure zero**, correctly assigning all predictive weight to true signals F1 and F2.
- **Takeaway:** Never deploy feature selection based on raw MDI without permutation verification.

Doc Guide: [API: permutation_importance](#)



Hands-on Lab 4: Tree MDI vs Permutation Importance

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

# 1. Load data and inject high-cardinality random noise column
data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, random_state=42)

rf_clf = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_tr, y_tr)

# 2. Compare MDI vs Permutation on Test Set
mdi_top3 = np.argsort(rf_clf.feature_importances_)[::-1][:3]
perm_out = permutation_importance(rf_clf, X_te, y_te, n_repeats=10, random_state=42)
perm_top3 = np.argsort(perm_out.importances_mean)[::-1][:3]

print("Top 3 by MDI: ", data.feature_names[mdi_top3])
print("Top 3 by Permutation:", data.feature_names[perm_top3])
```

Diagnostic Note:

Permutation importance evaluates on unseen test data, directly measuring true out-of-sample generalization.



Decision Matrix for Embedded Feature Selection Methods

Embedded Technique	Model Family	Metric Basis	Speed	Multicollinearity Behavior	Primary Production Hazard
Lasso (L_1 Penalty)	Linear / Logistic	Sparsity via $ \beta_j = 0$	Ultra-fast	Arbitrarily picks 1 from correlated set	Fails if features are unscaled
ElasticNet ($L_1 + L_2$)	Linear / Logistic	Grouped sparsity	Ultra-fast	Retains groups of correlated features	Extra hyperparameter (I_1_ratio)
Tree MDI (Gini)	Random Forest / GBDT	Impurity reduction ΔI	Zero extra cost	Splits importance among correlated trees	Severe bias toward high cardinality
Permutation Importance	100% Model-Agnostic	Out-of-sample score drop	Moderate	Correlated features dilute importance	Requires separate validation split

Executive Recommendation:

Use **ElasticNet** for high-dimensional linear problems with collinearity; use **Random Forest + Permutation Importance** for complex non-linear enterprise domains.

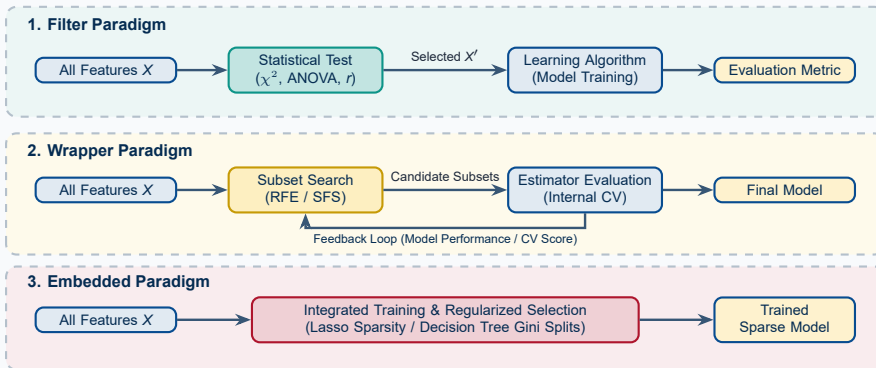


MODULE OUTLINE

Module 4: Advanced Paradigms, Hybrid Pipelines & Production Best Practices

Three Paradigms Architecture, Leakage Defense & Pipelines

The 3 Paradigms in Contrast: Architectural Topology



Doc Guide: [Scikit-Learn User Guide: Feature Selection Methods](#)



[IMPORTANT WARNING] How Multi-collinearity Distorts Selection

When two features X_1 and X_2 have correlation $\rho > 0.95$:

- **In Filter Methods:** Both are ranked high and retained, duplicating information.
- **In Lasso:** Arbitrarily selects one and zeroes out the other based on micro-fluctuations in sample noise.
- **In Random Forest:** The trees split randomly between X_1 and X_2 , diluting the importance of both by 50%!

Production Solution: Collinearity Triage

1. Compute pairwise correlation matrix $|R_{ij}|$.
2. Identify pairs exceeding threshold $\tau = 0.85$.
3. Prune one feature from each pair based on domain interpretability or higher univariate variance.
4. Alternatively, use **ElasticNet** which groups correlated features together.



[IMPORTANT WARNING] The Threat to Selection Integrity

Categorical features with high unique counts (e.g. Postal Code, Store ID, IP Subnet):

- **Chi-Squared (χ^2):** Degrees of freedom $df = (r - 1)(c - 1)$ expand rapidly. Aggregating noise deviations across numerous categories produces spuriously inflated χ^2 scores.
- **Tree Models:** Provides excessive split points, causing greedy trees to overfit and artificially inflating MDI.

Standardized Defense Protocol

- Never feed high-cardinality nominals directly to `chi2` or `SelectFromModel(RandomForest)`.
- **Remedy 1:** Target Encoding with empirical Bayes smoothing.
- **Remedy 2:** Group rare categories into an 'Other' bin before selection.
- **Remedy 3:** Verify rankings with out-of-fold Permutation Importance.



[IMPORTANT WARNING] Methodological Error: Selection Before Splitting

- Running SelectKBest, RFE, or Lasso on the entire dataset X before calling `train_test_split` leaks test distribution information into feature subset selection.
- In a dataset of pure random Gaussian noise with $P = 10,000$, chance correlations guarantee that the top 50 features correlate with y at $p < 0.01$.
- Validating on test yields 95% accuracy, but model fails completely on real data!

The Law of Production: Pipeline Integration

Raw Data $\longrightarrow$ Train / Test Split

Train Split $\longrightarrow$ Fit Pipeline(Selector $\rightarrow$ Estimator)

Test Split $\longrightarrow$ Transform Pipeline (Zero Target Peek)

Cross-validation workflows must fit the selector strictly on each training fold.

Doc Guide: [Scikit-Learn Guide: Preventing Data Leakage](#)

Mathematical Proof: Selection Leakage on Pure Noise

The Experiment (Freedman's Paradox)

Let $X \sim \mathcal{N}(0, I)$ be 100 samples of 1,000 independent noise variables. Target $y \sim \text{Bernoulli}(0.5)$ independent coin flips.

- True population correlation: $\rho(X_j, y) = 0, \forall j$.
- By extreme value theory, the sample maximum $|r_{\max}|$ across 1,000 noise features follows:

$$E[\max_j |r_j|] \approx \sqrt{\frac{2 \ln P}{N}} = \sqrt{\frac{2 \ln 1000}{100}} \approx 0.37$$

[IMPORTANT WARNING] Spurious Significance on Pure Noise

- Selecting the top 20 features prior to splitting yields an apparent $R^2 \approx 0.65$ ($p < 0.0001$) on completely random noise (Freedman's Paradox).
- **Conclusion:** A selector trained on test labels will find spurious statistical ghosts.
- **Strict Mandate:** Encapsulate selectors inside `sklearn.pipeline.Pipeline`.

In-Class Drill: Selection Leakage & Freedman's Paradox

Problem Statement: Noise Screening ($N = 50, P = 200$)

Pure Gaussian noise matrix $\mathbf{X}_{50 \times 200} \sim \mathcal{N}(0, 1)$ and random coin-flip labels $y \in \{0, 1\}$.

- True population correlation:
 $\rho_j \equiv 0.0, \forall j \in \{1, \dots, 200\}$.
- Extreme Value Theory bound:

$$E[|r_{\max}|] \approx \sqrt{\frac{2 \ln P}{N}} = \sqrt{\frac{2 \ln(200)}{50}}$$

Tasks for the Class (2 minutes):

1. Compute expected maximum spurious correlation ($\ln(200) \approx 5.30$).
2. Contrast: What is the true correlation ρ vs sample $r_{\max}$?
3. Predict: What happens if you filter top $k = 5$ features before cross-validation?

Student Worksheet: Fill in Noise Impact

Extreme value calculation: $\frac{2 \times 5.30}{50} = \frac{10.60}{50} = \dots$

Metric / Scenario	Theoretical Truth	Leaked Observation
True Population ρ	0.000	0.000
Expected Sample $ r_{\max} $	$\approx \sqrt{0.212} = \dots$	?
Apparent Cross-Val Score	$\approx 50\%$ (Random)	? (Artificially high)
Test Set Performance	$\approx 50\%$ (Random)	? (Collapse)

Guided Reflection: The Production Mandate

- Filtering before splitting creates an illusion of high predictive power.
- What single Scikit-learn object guarantees zero leakage across folds?
- *Detailed step-by-step solutions are in Master Solution Manual.*

Automated Production Solution: Scikit-Learn Pipeline



```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier

# 1. Split FIRST - strictly isolate test data
X_train, X_test, y_train, y_test = train_test_split(X_bench, y_bench, test_size=0.2, random_state=42)

# 2. Encapsulate Selection inside Pipeline
pipe = Pipeline([
    ("filter", SelectKBest(score_func=f_classif, k=8)),
    ("classifier", RandomForestClassifier(n_estimators=100, random_state=42))
])

# 3. Cross-Validation: SelectKBest is refitted on each fold's training split
cv_scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring="accuracy")
print(f"Leakage-Safe CV Score: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")

# 4. Final production fit and evaluation
pipe.fit(X_train, y_train)
print(f"Holdout Test Score: {pipe.score(X_test, y_test):.4f}")
```

Doc Guide: [Scikit-Learn User Guide: Pipeline and Composite Estimators](#)



Heterogeneous Selection with ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import SelectPercentile, chi2, f_classif
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder

num_cols = ["age", "income", "credit_score"]
cat_cols = ["education", "home_ownership"]

# Apply Chi2 to categorical features and ANOVA to numerical features
preprocessor = ColumnTransformer(transformers=[
    ("num_select", Pipeline([
        ("scale", MinMaxScaler()),
        ("anova", SelectPercentile(score_func=f_classif, percentile=60))
    ]), num_cols),
    ("cat_select", Pipeline([
        ("ohe", OneHotEncoder(sparse_output=False, handle_unknown="ignore")),
        ("chi2", SelectPercentile(score_func=chi2, percentile=50))
    ]), cat_cols)
])

production_pipeline = Pipeline([
    ("selection", preprocessor),
    ("model", LogisticRegression())
])
```

Doc Guide: [Scikit-Learn User Guide: ColumnTransformer for Heterogeneous Data](#)



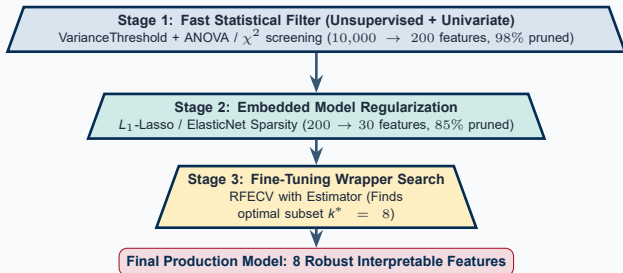
Hands-on Lab 5: Grand End-to-End Selection Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.feature\selection import VarianceThreshold, SelectFromModel
from sklearn.linear\model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report

# Grand 3-Stage Pipeline:
# Stage 1: Filter zero-variance constants
# Stage 2: Embedded L1-Lasso selector for sparsity
# Stage 3: High-capacity Gradient Boosting Estimator
grand_pipe = Pipeline([
    ("zero_var", VarianceThreshold(threshold=0.0)),
    ("scaler", StandardScaler()),
    ("l1_selector", SelectFromModel(estimator=LogisticRegression(penalty="l1", solver="liblinear", C=0.5))),
    ("gbdt_classifier", GradientBoostingClassifier(n_estimators=100, random_state=42))
])

grand_pipe.fit(X_train, y_train)
y_pred = grand_pipe.predict(X_test)
print(classification_report(y_test, y_pred))
```

Hybrid Feature Selection: The Multi-Stage Funnel



Production Funnel Principle:

Combine fast filter screening, embedded methods for structured pruning, and targeted wrappers for final subset refinement.

Doc Guide: [Scikit-Learn User Guide: Feature Selection](#)



Master Architectural Comparison Across All 4 Paradigms

Paradigm	Computational Cost	Model Agnostic?	Interaction Handling	Overfitting Risk	Best Suited Domain	Key Scikit-learn Tools
Filter	Ultra-Low $O(N \cdot P)$	Yes (100%)	None (Univariate)	Very Low	Ultra-high dimensions ($P > 10^4$)	SelectKBest, chi2, f_classif
Wrapper	Very High $O(2^P)$	Yes (SFS) / Semi (RFE)	Complete	High (needs strict CV)	Small/Medium tabular ($P \leq 50$)	RFE, RFECV, SequentialFeatureSelector
Embedded	Moderate (1 fit)	No (Model-Specific)	Model-dependent	Moderate	Regression / Trees with collinearity	LassoCV, SelectFromModel, MDI
Hybrid Funnel	Optimized	Semi-Agnostic	Staged	Low (controlled)	Industrial Enterprise ML Pipelines	Pipeline + Filter → Lasso → RFECV

The Golden Engineering Rule:

Never deploy raw feature selection outside an encapsulated Pipeline. Wrap all selectors and scalers to prevent data leakage and ensure 100

Summary & Actionable Feature Selection Checklist



1. Statistical Filter Methods

Use **ANOVA F-test** for numerical features and χ^2 for categorical features. Excellent for rapid initial screening, but blind to interactions.

2. Wrapper Methods (RFE / SFS)

RFE prunes least important features recursively. **RFECV** finds the mathematically optimal subset size k^* . Beware of $O(2^P)$ computational cost.

3. Embedded Methods

L_1 **Lasso** induces true zero sparsity (requires scaling). Validate tree **MDI** with out-of-sample **Permutation Importance** to eliminate cardinality bias.

4. Production Architecture

Always encapsulate feature selection inside Pipeline with ColumnTransformer. Performing selection strictly within training folds eliminates selection leakage.

Thank You!

Questions & Answers (Q&A)

Next Session: Lesson 09 — Dimensionality Reduction

Principal Component Analysis (PCA) · Truncated SVD · t-SNE & Manifolds